

beisbol-real-predicho

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Ing. en Desarrollo y Gestión de Software

Análisis Supervisado y No Supervisado

Implementación, evaluación y optimización de
modelos de aprendizaje automático

Alumno: Carlos Eduardo Morales

Grupo: IDGS14

Materia: Extracción de conocimiento en bases de datos

Profesor: Filiberto Ruiz

Índice

Introducción	3
1. Análisis Supervisado: Regresión con <code>beisbol.csv</code>	3
1.1. Descripción del problema	3
1.2. Justificación del algoritmo	4
1.3. Diseño del modelo: paso a paso del notebook	4
1.4. Evaluación del modelo	13
1.5. Optimización del modelo	14
1.6. Gráficas e interpretación	14
1.7. Repositorio del modelo	16
2. Análisis Supervisado: Clasificación con <code>breast_cancer.csv</code>	16
2.1. Descripción del problema	16
2.2. Justificación del algoritmo	17
2.3. Diseño del modelo: paso a paso del notebook	17
2.4. Evaluación y optimización	19
2.4.1. Resultados del modelo base	20
2.4.2. Resultados del modelo optimizado ($C = 10$)	20
2.4.3. Comparación entre modelos	21
2.5. Gráficas e interpretación	22
2.6. Repositorio del modelo	24
3. Análisis No Supervisado: Agrupación con <code>samsung.csv</code>	24
3.1. Descripción del problema	25
3.2. Justificación del algoritmo	25
3.3. Diseño del modelo: paso a paso del notebook	25
3.4. Optimización y métricas	27
3.4.1. Métricas del modelo final	28
3.5. Caracterización de los clusters	29
3.6. Gráficas e interpretación	30
3.7. Repositorio del modelo	32
4. Análisis No Supervisado: Reducción de Dimensionalidad con <code>comprar_alquilar.csv</code>	33
4.1. Descripción del problema	33
4.2. Justificación del algoritmo	33
4.3. Diseño del modelo: paso a paso del notebook	34
4.4. Selección del número de componentes	36
4.5. Interpretación de los componentes	37

4.6. Gráficas e interpretación	38
4.7. Repositorio del modelo	40
5. Conclusiones generales	41
Repositorio general	41

Introducción

El presente trabajo tiene como objetivo aplicar técnicas de aprendizaje automático sobre cuatro conjuntos de datos, cubriendo tanto el **análisis supervisado** como el **análisis no supervisado**. Para cada conjunto de datos se diseñó un procedimiento completo de preparación, entrenamiento, evaluación, optimización y visualización de resultados mediante Python y Jupyter Notebook.

En el análisis supervisado se desarrollaron dos modelos: uno de **regresión** para estimar el número de carreras en béisbol y otro de **clasificación** para identificar diagnósticos en un conjunto de datos de cáncer de mama. En el análisis no supervisado se implementó un modelo de **agrupación** para estudiar los registros del conjunto de Samsung y un método de **reducción de dimensionalidad** para representar el conjunto de datos de compra/alquiler de vivienda.

La selección de los algoritmos se realizó considerando la naturaleza de cada problema, el tipo de variables disponibles, la facilidad de interpretación y la posibilidad de evaluar y optimizar los modelos mediante métricas y búsqueda de hiperparámetros.

Objetivo general

Desarrollar y evaluar modelos de aprendizaje automático capaces de extraer patrones y realizar predicciones o representaciones útiles a partir de los conjuntos de datos proporcionados, justificando las decisiones tomadas durante el proceso.

1. Análisis Supervisado: Regresión con `beisbol.csv`

1.1 Descripción del problema

El conjunto de datos `beisbol.csv` contiene información de diferentes equipos de béisbol. Sus variables principales son:

Cuadro 1: Variables utilizadas en el problema de regresión.

Variable	Tipo	Descripción
<code>equipos</code>	Categórica	Nombre del equipo
<code>bateos</code>	Numérica	Cantidad de bateos registrados
<code>runs</code>	Numérica	Carreras registradas

El objetivo consiste en **predecir la cantidad de carreras (runs) a partir de la cantidad de bateos (bateos)**. La variable `equipos` se considera identificadora y no se utiliza como predictor numérico.

1.2 Justificación del algoritmo

Se seleccionó inicialmente el algoritmo de **Regresión Lineal** debido a que el problema cuenta con una variable independiente numérica y una variable objetivo numérica continua. Además, resulta apropiado como modelo base porque permite analizar de forma sencilla la relación entre la cantidad de bateos y las carreras obtenidas.

La regresión lineal busca aproximar la relación entre las variables mediante una ecuación de la forma:

$$\hat{y} = \beta_0 + \beta_1 x$$

donde $\hat{y}$ representa las carreras predichas, x representa los bateos, β_0 es la intersección y β_1 es el coeficiente que describe el cambio esperado en las carreras por cada unidad adicional de bateo.

Para la etapa de optimización también se consideran modelos regularizados como **Ridge** y **Lasso**. Estos modelos permiten controlar la magnitud de los coeficientes y comparar su desempeño con el modelo lineal base.

1.3 Diseño del modelo: paso a paso del notebook

El desarrollo del modelo se realizó de manera incremental, comenzando con la exploración y preparación del conjunto de datos y terminando con la selección y persistencia del mejor modelo. Debido a que el conjunto contiene únicamente 30 registros, se utilizó una división de entrenamiento y prueba del 80/20 y se complementó la evaluación mediante validación cruzada.

1. Importación de librerías.

Se utilizaron librerías para el manejo de datos, visualización, construcción de modelos, evaluación y almacenamiento del modelo entrenado.

```
1 import os
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
```

```
5 import seaborn as sns
6
7 from sklearn.model_selection import train_test_split
8 from sklearn.model_selection import GridSearchCV, cross_val_score
9
10 from sklearn.linear_model import LinearRegression, Ridge, Lasso
11
12 from sklearn.metrics import (
13     mean_absolute_error,
14     mean_squared_error,
15     r2_score
16 )
17
18 import joblib
```

2. Carga del conjunto de datos.

El archivo `beisbol.csv` se cargó mediante Pandas. Como primera comprobación se revisó el número de registros y variables.

```
1 ruta = "../datasets/beisbol.csv"
2
3 df = pd.read_csv(ruta)
4
5 print("Dataset cargado correctamente.")
6 print(f"Filas: {df.shape[0]}")
7 print(f"Columnas: {df.shape[1]}")
```

El conjunto contiene **30 registros** y **4 columnas**: `Unnamed: 0`, `equipos`, `bateos` y `runs`.

3. Exploración y limpieza inicial.

Se revisó la estructura del DataFrame mediante `info()`, las estadísticas descriptivas y los valores nulos.

```
1 df.info()
2 df.describe()
3
4 print("Valores nulos:")
5 print(df.isnull().sum())
```

La revisión mostró que no existen valores nulos en las variables. También se identificó la columna `Unnamed: 0`, generada como índice durante el almacenamiento

del archivo CSV. Debido a que esta columna no representa una característica del problema, fue eliminada.

```
1 df = df.drop(columns=["Unnamed: 0"])
2
3 print("Columnas despues de la limpieza:")
4 print(df.columns.tolist())
```

Después de la limpieza quedaron las columnas **equipos**, **bateos** y **runs**.

4. Selección de variables.

Se estableció **bateos** como variable independiente y **runs** como variable objetivo. La columna **equipos** se mantuvo únicamente como identificador descriptivo y no se utilizó como predictor.

```
1 X = df[["bateos"]]
2 y = df["runs"]
3
4 print("Forma de X:", X.shape)
5 print("Forma de y:", y.shape)
```

El resultado fue:

```
1 Forma de X: (30, 1)
2 Forma de y: (30,)
```

Por lo tanto, el modelo utiliza una sola variable predictora para estimar la cantidad de carreras.

5. Visualización inicial.

Antes de entrenar el modelo se generó una gráfica de dispersión para observar la relación entre **bateos** y **runs**.

```
1 plt.figure(figsize=(10, 6))
2
3 sns.scatterplot(
4     data=df,
5     x="bateos",
6     y="runs",
7     s=90
8 )
9
10 plt.title("Relacin entre bateos y carreras")
```

```
11 plt.xlabel("Bateos")
12 plt.ylabel("Carreras (runs)")
13 plt.grid(alpha=0.3)
14 plt.tight_layout()
15
16 plt.savefig(
17     "../figuras/beisbol_dispersion.png",
18     dpi=300,
19     bbox_inches="tight"
20 )
21
22 plt.show()
```

La visualización permitió comprobar que existe una tendencia general positiva entre ambas variables; sin embargo, los registros presentan una dispersión considerable alrededor de dicha tendencia, por lo que desde esta etapa no se observa una relación lineal suficientemente precisa como para garantizar predicciones exactas.

6. Separación de datos.

El conjunto se dividió en datos de entrenamiento y prueba utilizando una proporción de 80/20. Debido al tamaño reducido del conjunto, se estableció `random_state=42` para garantizar que la partición fuera reproducible.

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     X,
3     y,
4     test_size=0.2,
5     random_state=42
6 )
7
8 print("Datos de entrenamiento:", X_train.shape[0])
9 print("Datos de prueba:", X_test.shape[0])
```

La división produjo **24 registros para entrenamiento y 6 para prueba.**

7. Entrenamiento del modelo base.

Como primera aproximación se implementó una Regresión Lineal simple.

```
1 modelo_lineal = LinearRegression()
2
3 modelo_lineal.fit(
```

```
4     X_train,  
5     y_train  
6 )
```

La ecuación obtenida por el modelo fue:

$$\hat{runs} = -2981.5172 + 0.66685(bateos)$$

El coeficiente positivo indica que el modelo estima un incremento aproximado de 0.667 carreras por cada bateo adicional. Sin embargo, este coeficiente por sí solo no determina la calidad del modelo, por lo que fue necesario evaluar sus predicciones.

8. Generación de predicciones.

Se calcularon las predicciones utilizando exclusivamente los datos de prueba.

```
1 y_pred = modelo_lineal.predict(X_test)  
2  
3 comparacion = pd.DataFrame({  
4     "Valor real": y_test.values,  
5     "Valor predicho": y_pred  
6 })  
7  
8 comparacion
```

Las predicciones mostraron diferencias importantes respecto a algunos valores reales. Por ejemplo, una observación con 570 carreras fue estimada en aproximadamente 676.83, mientras que otra con 615 carreras fue estimada en aproximadamente 751.51.

9. Evaluación del modelo base.

Se calcularon MAE, RMSE y R^2 para medir el desempeño del modelo.

```
1 mae = mean_absolute_error(y_test, y_pred)  
2  
3 rmse = np.sqrt(  
4     mean_squared_error(y_test, y_pred)  
5 )  
6  
7 r2 = r2_score(  
8     y_test,
```

```
9     y_pred
10 )
11
12 print(f"MAE: {mae:.4f}")
13 print(f"RMSE: {rmse:.4f}")
14 print(f"R: {r2:.4f}")
```

La Regresión Lineal obtuvo:

```
1 MAE: 51.6116
2 RMSE: 72.2494
3 R: -0.5175
```

El valor negativo de R^2 indica que el modelo presenta un desempeño inferior al de una predicción basada únicamente en el promedio de la variable objetivo. Por lo tanto, la utilización de `bateos` como único predictor no permite explicar adecuadamente la variabilidad observada en `runs`.

10. Validación cruzada.

Para verificar si el desempeño obtenido dependía demasiado de una sola división de los datos, se utilizó validación cruzada de cinco particiones.

```
1 scores = cross_val_score(
2     modelo_lineal,
3     X,
4     y,
5     cv=5,
6     scoring="r2"
7 )
8
9 print("Resultados de R por particin:")
10 print(scores)
11
12 print(f"R promedio: {scores.mean():.4f}")
13 print(f"Desviacin estndar: {scores.std():.4f}")
```

Los resultados fueron:

```
1 [-1.06322413 -0.55530097 -0.09209403
2  -0.02235244 -1.15050052]
3
4 R promedio: -0.5767
5 Desviacin estndar: 0.4709
```

El promedio negativo obtenido mediante las cinco particiones confirma que el bajo desempeño no corresponde únicamente a los seis registros utilizados en la prueba. El modelo presenta un comportamiento limitado de manera consistente al utilizar únicamente bateos.

11. Optimización mediante Ridge y Lasso.

Como alternativa al modelo lineal base se implementaron Ridge y Lasso. Ambos métodos incorporan regularización mediante el hiperparámetro α , por lo que se realizó una búsqueda de diferentes valores utilizando `GridSearchCV`.

```
1 parametros = {  
2     "alpha": [0.01, 0.1, 1, 10, 100, 1000]  
3 }  
4  
5 ridge = Ridge()  
6  
7 grid_ridge = GridSearchCV(  
8     ridge,  
9     parametros,  
10    cv=5,  
11    scoring="r2"  
12 )  
13  
14 grid_ridge.fit(X_train, y_train)
```

Para Lasso se utilizó el mismo procedimiento:

```
1 lasso = Lasso()  
2  
3 grid_lasso = GridSearchCV(  
4     lasso,  
5     parametros,  
6     cv=5,  
7     scoring="r2"  
8 )  
9  
10 grid_lasso.fit(X_train, y_train)
```

La búsqueda determinó:

- **Ridge:** $\alpha = 1000$, con un R^2 promedio de validación de 0.2086.
- **Lasso:** $\alpha = 100$, con un R^2 promedio de validación de 0.2095.

Estos valores corresponden a la validación cruzada realizada durante la búsqueda de hiperparámetros sobre el conjunto de entrenamiento. Para determinar el desempeño final de cada modelo se realizó posteriormente una evaluación independiente sobre el conjunto de prueba.

12. Comparación de modelos.

Se calcularon las mismas métricas para los tres modelos y se almacenaron en una tabla para facilitar su comparación.

```

1 def evaluar_modelo(nombre, y_real, y_pred):
2     mae = mean_absolute_error(y_real, y_pred)
3     rmse = np.sqrt(mean_squared_error(y_real, y_pred))
4     r2 = r2_score(y_real, y_pred)
5
6     return {
7         "Modelo": nombre,
8         "MAE": mae,
9         "RMSE": rmse,
10        "R": r2
11    }

```

Los resultados finales sobre el conjunto de prueba fueron:

Cuadro 2: Comparación de los modelos de regresión.

Modelo	MAE	RMSE	R^2
Regresión Lineal	51.6116	72.2494	-0.5175
Ridge	51.6004	72.1864	-0.5148
Lasso	51.5709	72.0239	-0.5080

El modelo **Lasso** obtuvo el mejor resultado en las tres métricas consideradas: presentó el menor MAE, el menor RMSE y el mayor R^2 . No obstante, la diferencia respecto a los otros modelos fue reducida.

13. Selección del modelo final.

El modelo final se seleccionó automáticamente tomando aquel que presentó el mayor R^2 sobre el conjunto de prueba.

```

1 resultados = pd.DataFrame([
2     evaluar_modelo(
3         "Regresin Lineal",

```

```
4         y_test,  
5         pred_lineal  
6     ),  
7     evaluar_modelo(  
8         "Ridge",  
9         y_test,  
10        pred_ridge  
11    ),  
12    evaluar_modelo(  
13        "Lasso",  
14        y_test,  
15        pred_lasso  
16    )  
17 ])  
18  
19 mejor_modelo_nombre = resultados.loc[  
20     resultados["R"].idxmax(),  
21     "Modelo"  
22 ]
```

El modelo seleccionado fue **Lasso** con $\alpha = 100$.

14. Persistencia del modelo.

Finalmente, el modelo seleccionado se guardó mediante `joblib` para integrarlo al repositorio del proyecto.

```
1 os.makedirs(  
2     "../models/regression",  
3     exist_ok=True  
4 )  
5  
6 joblib.dump(  
7     mejor_modelo,  
8     "../models/regression/"  
9     "mejor_modelo_béisbol.pkl"  
10 )
```

El archivo generado corresponde al modelo de regresión utilizado como resultado final del ejercicio.

1.4 Evaluación del modelo

Para evaluar los modelos se utilizaron MAE, MSE, RMSE y R^2 . Estas métricas permiten analizar tanto el tamaño promedio de los errores como la capacidad del modelo para explicar la variabilidad de la variable objetivo.

- **MAE (Mean Absolute Error):** representa el error absoluto promedio de las predicciones. Un valor menor indica que, en promedio, las predicciones se encuentran más cerca de los valores reales.
- **MSE (Mean Squared Error):** calcula el promedio de los errores al cuadrado, por lo que penaliza de manera más fuerte las predicciones con errores grandes.
- **RMSE (Root Mean Squared Error):** corresponde a la raíz cuadrada del MSE y mantiene las mismas unidades que **runs**.
- R^2 : indica qué proporción de la variabilidad de la variable objetivo es explicada por el modelo. Un valor cercano a 1 representa un buen ajuste, mientras que un valor negativo indica que el modelo presenta un desempeño inferior al de una predicción basada en el promedio.

Para este problema, el modelo preferible es aquel que presente menores valores de MAE, MSE y RMSE y un mayor valor de R^2 .

Resultados de regresión

Modelo final: Lasso con $\alpha = 100$.

MAE: 51.5709

MSE: 5187.4390

RMSE: 72.0239

R^2 : -0.5080

Aunque Lasso fue el modelo con mejor desempeño relativo entre las tres alternativas, sus métricas muestran que el modelo todavía presenta una capacidad predictiva limitada. La mejora sobre la Regresión Lineal fue pequeña, por lo que los resultados sugieren que **bateos** por sí solo no contiene suficiente información para realizar una predicción precisa de **runs**. Para obtener un modelo más preciso sería necesario incorporar otras variables relacionadas con el desempeño ofensivo de los equipos.

1.5 Optimización del modelo

La optimización se realizó mediante `GridSearchCV`, evaluando diferentes valores del hiperparámetro α para Ridge y Lasso utilizando cinco particiones de validación cruzada.

```
1 parametros = {  
2     "alpha": [0.01, 0.1, 1, 10, 100, 1000]  
3 }
```

La búsqueda encontró $\alpha = 1000$ para Ridge y $\alpha = 100$ para Lasso. Sin embargo, la selección final se realizó comparando el desempeño de cada modelo sobre el conjunto de prueba independiente.

Justificación de la optimización

La optimización permitió comparar distintas intensidades de regularización y evitar depender exclusivamente de la primera Regresión Lineal entrenada. Lasso obtuvo el mejor resultado final, aunque la mejora fue reducida. Esto demuestra que la regularización ayuda ligeramente al modelo, pero también evidencia que la principal limitación se encuentra en utilizar una única variable predictora.

1.6 Gráficas e interpretación

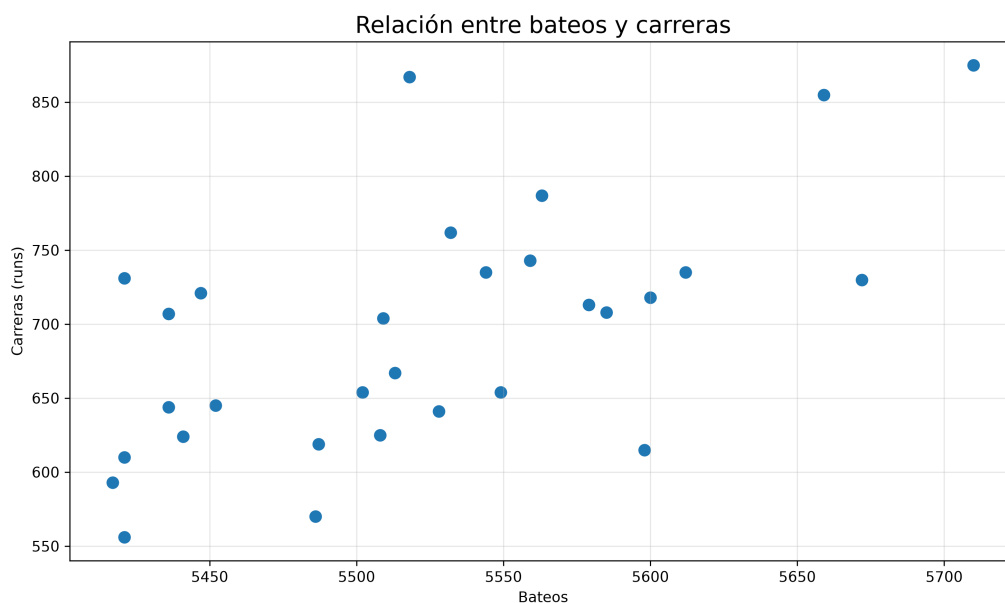


Figura 1: Relación entre la cantidad de bateos y las carreras registradas en los 30 equipos del conjunto de datos.

La gráfica de dispersión muestra una **tendencia general positiva**: los equipos con una mayor cantidad de bateos tienden a registrar también una mayor cantidad de carreras. Sin embargo, los puntos no se concentran estrechamente alrededor de una única línea, sino que presentan una dispersión considerable. Esta variabilidad explica por qué la cantidad de bateos, aunque relacionada con las carreras, no resulta suficiente por sí sola para realizar predicciones precisas.

La dispersión observada es consistente con los resultados obtenidos posteriormente: el modelo lineal presentó un R^2 negativo y un error considerable, indicando que existen otros factores que influyen en la cantidad de carreras registradas y que no están representados en el predictor utilizado.

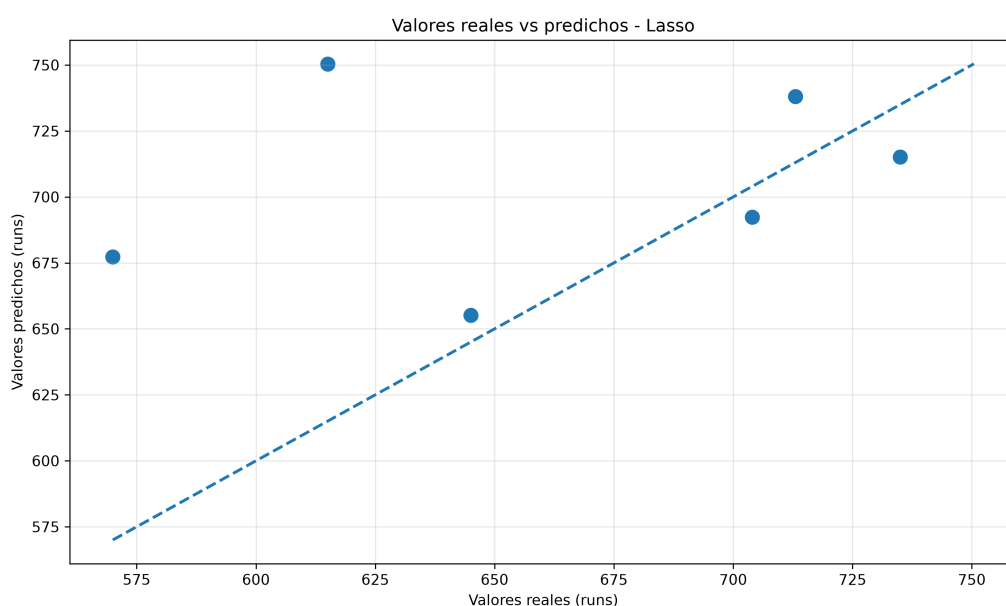


Figura 2: Comparación de las carreras reales frente a las carreras predichas por el modelo Lasso seleccionado. La diagonal representa una predicción ideal.

La gráfica compara las carreras reales del conjunto de prueba con las predicciones generadas por el modelo Lasso. La línea diagonal representa el escenario ideal, en el cual cada predicción coincide exactamente con el valor real.

En los resultados obtenidos, varios puntos presentan una separación considerable respecto de la diagonal. En particular, se observan predicciones que sobreestiman de manera importante algunos valores reales, lo cual coincide con el MAE de **51.5709** y el RMSE de **72.0239**. La distribución de los puntos confirma visualmente que el modelo Lasso, aunque obtuvo el mejor desempeño entre las alternativas probadas, todavía presenta errores importantes al predecir las carreras.

En conjunto, ambas gráficas permiten relacionar el comportamiento visual de los datos con las métricas obtenidas: existe una relación positiva entre los bateos y las carreras,

pero la dispersión de los registros limita la capacidad de un modelo basado exclusivamente en `bateos`.

1.7 Repositorio del modelo

El código fuente, notebook y modelo obtenido se integran en el siguiente repositorio:

<https://github.com/CarlitrossMM/analisis-supervisado-no-supervisado>

La versión final del modelo de regresión se almacena en la carpeta:

`models/regression/`

2. Análisis Supervisado: Clasificación con `breast_cancer.csv`

2.1 Descripción del problema

El conjunto de datos `breast_cancer.csv` contiene 569 registros correspondientes a muestras de cáncer de mama. Cada registro incluye 30 características numéricas derivadas de imágenes digitalizadas y una variable objetivo llamada `diagnosis`, que indica si la muestra es benigna (B) o maligna (M).

La columna `id` actúa únicamente como identificador y no aporta información diagnóstica, por lo que se elimina antes del modelado. La variable `diagnosis` se codifica numéricamente de la siguiente forma:

- B (benigno) $\rightarrow 0$
- M (maligno) $\rightarrow 1$

La distribución de clases es la siguiente:

- Clase 0 (benigno): 357 registros (62,74 %)
- Clase 1 (maligno): 212 registros (37,26 %)

El objetivo del ejercicio consiste en construir un modelo de clasificación binaria capaz de predecir el diagnóstico a partir de las 30 variables numéricas.

2.2 Justificación del algoritmo

Se seleccionó **Regresión Logística** porque el problema es de clasificación binaria y las variables predictoras son numéricas. Este algoritmo permite estimar la probabilidad de pertenecer a la clase positiva (maligno), es interpretable a través de sus coeficientes y admite regularización mediante el hiperparámetro C .

Dado que las características presentan escalas diferentes, se incorporó **StandardScaler** dentro de un **Pipeline**. De esta forma, el escalamiento se ajusta únicamente con los datos de entrenamiento, evitando fuga de información tanto en el entrenamiento inicial como durante la validación cruzada.

La optimización se realizó mediante **GridSearchCV**, explorando distintos valores de C y utilizando el F1-score como criterio de selección, al tratarse de una métrica que equilibra precisión y sensibilidad.

2.3 Diseño del modelo: paso a paso del notebook

A continuación se describen las etapas principales desarrolladas en el notebook, junto con los fragmentos de código más relevantes.

1. Carga e inspección de los datos.

```
1 ruta = "../datasets/breast-cancer.csv"
2 df = pd.read_csv(ruta)
3
4 print(f"Filas: {df.shape[0]}")
5 print(f"Columnas: {df.shape[1]}")
6 print(df.columns.tolist())
```

El dataset contiene 569 filas y 32 columnas (incluyendo `id` y `diagnosis`).

2. Limpieza y preparación. Se elimina la columna identificadora y se codifica la variable objetivo.

```
1 df = df.drop(columns=["id"])
2
3 df["diagnosis"] = df["diagnosis"].map({"B": 0, "M": 1})
4
5 print(df["diagnosis"].value_counts())
```

3. Separación de variables.

```
1 X = df.drop(columns=["diagnosis"])
2 y = df["diagnosis"]
3
4 print("Forma de X:", X.shape)
5 print("Forma de y:", y.shape)
```

Se obtienen 30 variables predictoras y una variable objetivo binaria.

4. División estratificada entrenamiento/prueba.

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y,
3     test_size=0.20,
4     random_state=42,
5     stratify=y
6 )
7
8 print("Datos de entrenamiento:", X_train.shape[0])
9 print("Datos de prueba:", X_test.shape[0])
```

La división estratificada conserva la proporción de clases: 455 registros para entrenamiento y 114 para prueba.

5. Construcción del Pipeline.

```
1 pipeline = Pipeline([
2     ("scaler", StandardScaler()),
3     ("logistic", LogisticRegression(max_iter=5000))
4 ])
```

El uso del Pipeline garantiza que el escalamiento se realice de forma correcta en cada partición de validación cruzada.

6. Entrenamiento del modelo base y generación de predicciones.

```
1 pipeline.fit(X_train, y_train)
2
3 y_pred = pipeline.predict(X_test)
4 y_prob = pipeline.predict_proba(X_test)[: , 1]
```

7. Evaluación del modelo base.

Se calculan Accuracy, Precision, Recall, F1-score y AUC-ROC.

8. Optimización mediante GridSearchCV.

```
1 param_grid = {  
2     "logistic__C": [0.001, 0.01, 0.1, 1, 10, 100]  
3 }  
4  
5 grid_search = GridSearchCV(  
6     estimator=pipeline,  
7     param_grid=param_grid,  
8     cv=5,  
9     scoring="f1",  
10    n_jobs=-1  
11 )  
12  
13 grid_search.fit(X_train, y_train)  
14  
15 print("Mejores hiperparametros:", grid_search.best_params_)  
16 print("Mejor F1-score promedio:", grid_search.best_score_)
```

El mejor valor encontrado fue $C = 10$, con un F1-score promedio de validación cruzada de 0,9611.

9. Evaluación del modelo optimizado y selección del modelo final.

10. Persistencia del modelo.

```
1 os.makedirs("../models/classification", exist_ok=True)  
2 model_path = "../models/classification/  
3     breast_cancer_logistic_regression.joblib"  
4 joblib.dump(best_model, model_path)
```

2.4 Evaluación y optimización

Para evaluar el clasificador se utilizaron las siguientes métricas:

- **Accuracy:** proporción global de predicciones correctas.
- **Precision:** proporción de predicciones positivas que realmente pertenecen a la clase positiva.
- **Recall:** capacidad del modelo para identificar correctamente los casos positivos (malignos).

- **F1-score:** media armónica entre Precision y Recall.
- **AUC-ROC:** mide la capacidad global del modelo para distinguir entre las dos clases.

En un problema médico, el **Recall** resulta especialmente relevante, ya que un falso negativo implica clasificar como benigno un caso que realmente es maligno.

2.4.1 Resultados del modelo base

Métrica	Valor
Accuracy	0,9649
Precision	0,9750
Recall	0,9286
F1-score	0,9512
AUC-ROC	0,9960

Matriz de confusión del modelo base:

$$\begin{bmatrix} 71 & 1 \\ 3 & 39 \end{bmatrix}$$

- Verdaderos negativos (TN): 71
- Falsos positivos (FP): 1
- Falsos negativos (FN): 3
- Verdaderos positivos (TP): 39

2.4.2 Resultados del modelo optimizado ($C = 10$)

Métrica	Valor
Accuracy	0,9737
Precision	0,9756
Recall	0,9524
F1-score	0,9639
AUC-ROC	0,9864

Matriz de confusión del modelo optimizado:

$$\begin{bmatrix} 71 & 1 \\ 2 & 40 \end{bmatrix}$$

- Verdaderos negativos (TN): 71
- Falsos positivos (FP): 1
- Falsos negativos (FN): 2
- Verdaderos positivos (TP): 40

2.4.3 Comparación entre modelos

Métrica	Modelo base	Modelo optimizado	Diferencia
Accuracy	0,9649	0,9737	+0,0088
Precision	0,9750	0,9756	+0,0006
Recall	0,9286	0,9524	+0,0238
F1-score	0,9512	0,9639	+0,0126
AUC-ROC	0,9960	0,9864	−0,0096

El modelo optimizado mejora Accuracy, Precision, Recall y F1-score. La mejora más relevante se observa en el Recall, que pasa de 0,9286 a 0,9524, lo que se traduce en la reducción de un falso negativo (de 3 a 2). Este comportamiento es favorable para el problema, ya que disminuye el número de casos malignos no detectados.

El AUC-ROC desciende ligeramente de 0,9960 a 0,9864. Aunque la reducción debe señalarse, el valor permanece elevado y el modelo conserva una alta capacidad de discriminación. Por tanto, la optimización no mejora todas las métricas de forma uniforme, pero sí las métricas de clasificación consideradas prioritarias (especialmente Recall y F1-score).

Por estos motivos, se seleccionó como modelo final el pipeline optimizado con $C = 10$.

2.5 Gráficas e interpretación

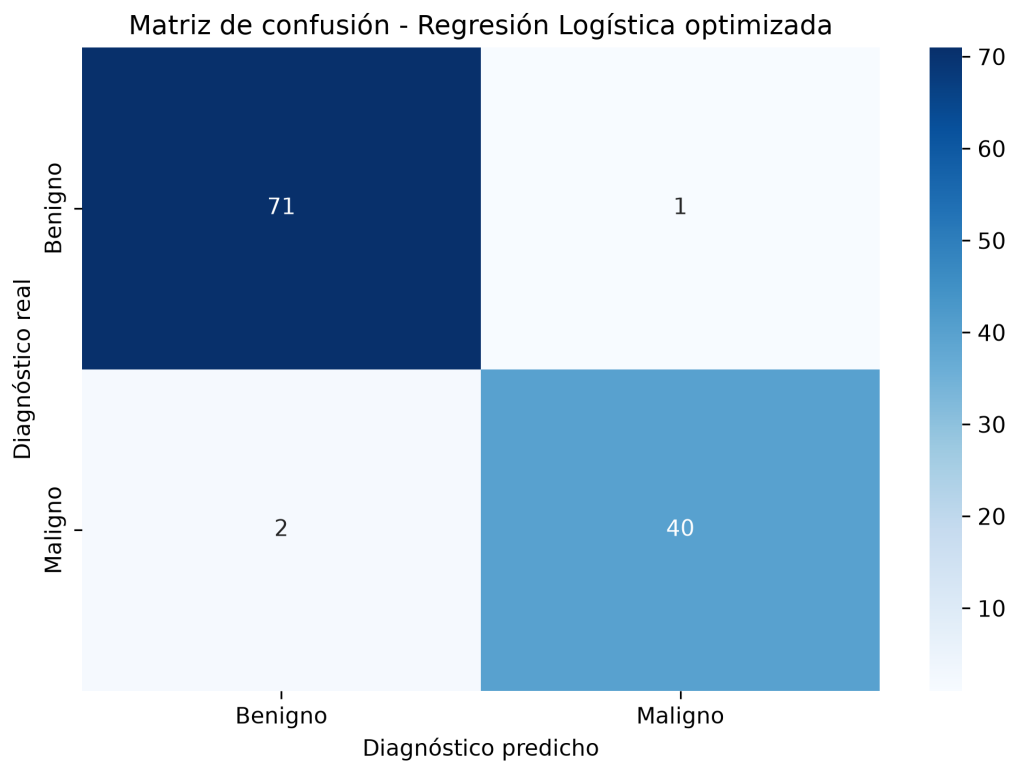


Figura 3: Matriz de confusión del modelo final (Regresión Logística optimizada).

La matriz de confusión del modelo optimizado muestra 71 verdaderos negativos, 40 verdaderos positivos, 1 falso positivo y 2 falsos negativos. La reducción de un falso negativo respecto al modelo base confirma la mejora en la capacidad de detectar casos malignos.

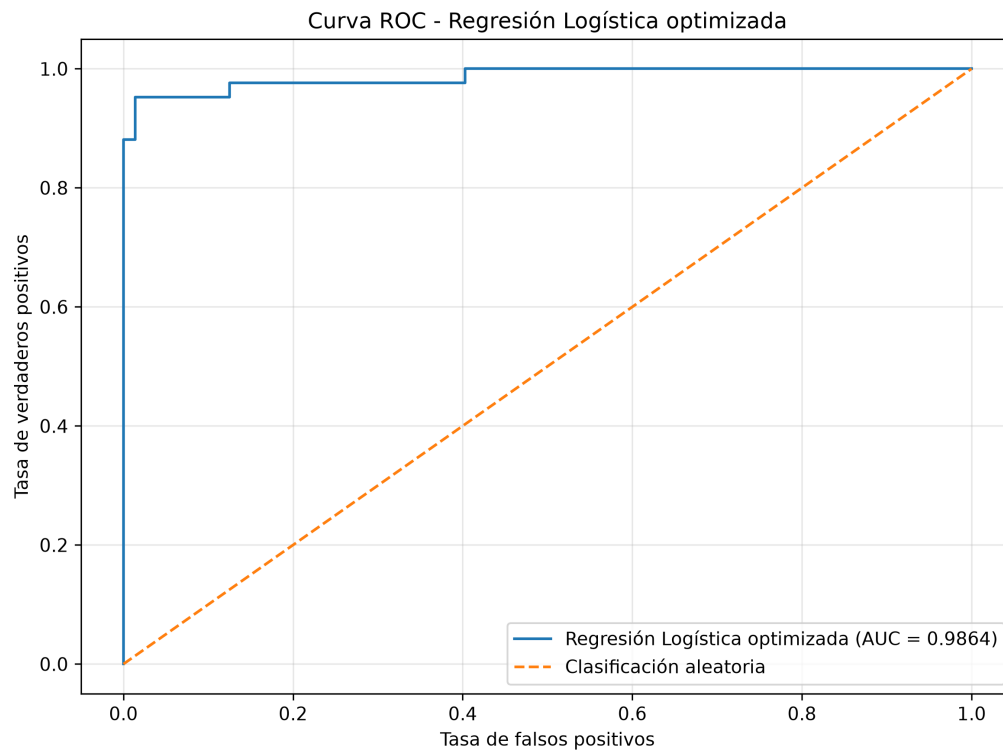


Figura 4: Curva ROC del modelo final de Regresión Logística optimizada.

La curva ROC del modelo optimizado se ubica claramente por encima de la diagonal de clasificación aleatoria y alcanza un AUC de 0,9864. Este valor indica una elevada capacidad de discriminación entre las clases benigna y maligna, aunque no debe interpretarse como un desempeño perfecto.

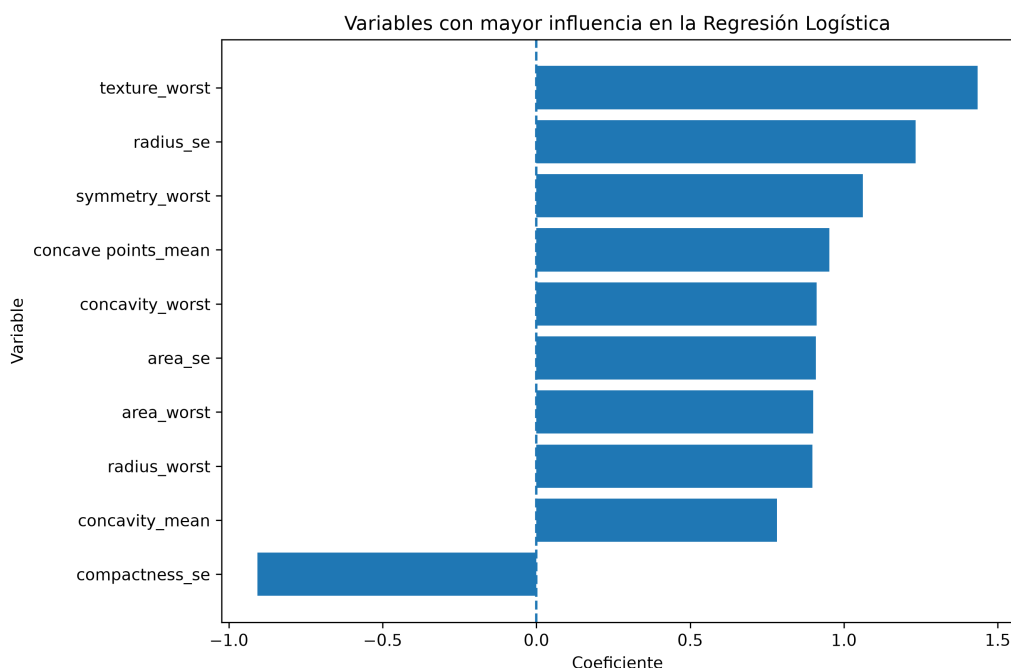


Figura 5: Diez variables con mayor magnitud de coeficiente en el modelo de Regresión Logística.

Las variables con mayor influencia (en valor absoluto) fueron, entre otras, `texture_worst`, `radius_se`, `symmetry_worst`, `concave points_mean` y `concavity_worst`. La mayoría de estos coeficientes son positivos, lo que indica que valores más altos de estas características aumentan la probabilidad de clasificar la muestra como maligna. Destaca el coeficiente negativo de `compactness_se`, que favorece la clasificación hacia la clase benigna. Dado que las variables fueron estandarizadas, los coeficientes pueden compararse de forma razonable en magnitud.

2.6 Repositorio del modelo

El código completo del notebook y el modelo final se encuentran disponibles en:

<https://github.com/CarlitrossMM/analisis-supervisado-no-supervisado>

El modelo de clasificación se almacena en la ruta:

`models/classification/breast_cancer_logistic_regression.joblib`

3. Análisis No Supervisado: Agrupación con `samsung.csv`

3.1 Descripción del problema

El conjunto de datos `samsung.csv` contiene 2850 observaciones diarias relacionadas con el comportamiento bursátil de Samsung. Las variables disponibles son:

Variable	Tipo	Uso en el ejercicio
Date	Fecha	Referencia temporal (no utilizada en el clustering)
Close	Numérica	Precio de cierre
Volume	Numérica	Volumen negociado

El objetivo consiste en identificar grupos de días con comportamientos similares a partir de las variables `Close` y `Volume`. La variable `Date` no se incorpora directamente al algoritmo de agrupación, ya que representa información temporal y no una característica numérica del comportamiento bursátil sobre la cual se desee calcular distancias.

3.2 Justificación del algoritmo

Se seleccionó **K-Means** porque el problema corresponde a aprendizaje no supervisado y se busca descubrir grupos de observaciones similares sin contar con una variable objetivo previamente definida. K-Means agrupa las observaciones minimizando la suma de las distancias cuadráticas entre cada punto y el centroide de su grupo:

$$\min \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

donde C_i representa el conjunto de observaciones del grupo i y μ_i su centroide.

Dado que `Close` y `Volume` se encuentran en escalas muy diferentes, se aplicó `StandardScaler` antes del agrupamiento. Para integrar ambos pasos de forma reproducible, se construyó un **Pipeline** compuesto por el escalador y el modelo K-Means.

3.3 Diseño del modelo: paso a paso del notebook

A continuación se describen las etapas principales desarrolladas en el notebook, junto con los fragmentos de código más relevantes.

1. Carga y exploración de los datos.

```
1 df = pd.read_csv("../data/samsung.csv")
2
3 print("Dimensiones del dataset:", df.shape)
```

```
4 print(df.info())
5 print(df.isnull().sum())
6 print(df.duplicated().sum())
7 print(df.describe())
```

El dataset contiene 2850 registros, no presenta valores nulos ni filas duplicadas.

2. **Conversión de la fecha.** La columna `Date` se convierte al tipo `datetime` para facilitar su tratamiento, aunque no se utiliza como variable de entrada del clustering.

```
1 df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)
```

3. **Selección de variables.**

```
1 X = df[["Close", "Volume"]].copy()
2 print("Forma de X:", X.shape)
```

4. **Estandarización.**

```
1 scaler = StandardScaler()
2 X_scaled = scaler.fit_transform(X)
```

5. **Búsqueda del número óptimo de clusters.** Se evaluaron valores de k desde 2 hasta 10 utilizando tres métricas: Silhouette Score, índice Calinski-Harabasz e índice Davies-Bouldin.

```
1 k_values = range(2, 11)
2
3 silhouette_scores = []
4 calinski_scores = []
5 davies_scores = []
6
7 for k in k_values:
8     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
9     labels = kmeans.fit_predict(X_scaled)
10
11     silhouette_scores.append(silhouette_score(X_scaled, labels))
12     calinski_scores.append(calinski_harabasz_score(X_scaled, labels))
13     davies_scores.append(davies_bouldin_score(X_scaled, labels))
14
15 resultados_k = pd.DataFrame({
16     "k": list(k_values),
```

```
17     "Silhouette": silhouette_scores,  
18     "Calinski-Harabasz": calinski_scores,  
19     "Davies-Bouldin": davies_scores  
20 })
```

6. Selección de k y entrenamiento del modelo final.

```
1 kmeans_final = KMeans(  
2     n_clusters=3,  
3     random_state=42,  
4     n_init=10  
5 )  
6 clusters = kmeans_final.fit_predict(X_scaled)  
7 df["Cluster"] = clusters
```

7. Construcción del Pipeline final y persistencia.

```
1 final_pipeline = Pipeline([  
2     ("scaler", StandardScaler()),  
3     ("kmeans", KMeans(  
4         n_clusters=3,  
5         random_state=42,  
6         n_init=10  
7     ))  
8 ])  
9  
10 final_pipeline.fit(X)  
11  
12 os.makedirs("../models/clustering", exist_ok=True)  
13 joblib.dump(  
14     final_pipeline,  
15     "../models/clustering/samsung_kmeans.joblib"  
16 )
```

3.4 Optimización y métricas

Se probaron valores de k entre 2 y 10. Las métricas utilizadas fueron:

- **Silhouette Score:** mayor valor es mejor (cohesión interna y separación entre grupos).

- **Calinski-Harabasz:** mayor valor es mejor (relación entre dispersión inter-cluster e intra-cluster).
- **Davies-Bouldin:** menor valor es mejor (separación entre grupos y dispersión interna).

Los resultados obtenidos fueron:

k	Silhouette	Calinski-Harabasz	Davies-Bouldin
2	0,4461	2635,68	0,8792
3	0,5013	3520,66	0,6852
4	0,4774	3787,42	0,7165
5	0,4602	3705,37	0,7442
6	0,4552	3556,61	0,7385
7	0,4321	3461,41	0,8164
8	0,4271	3508,16	0,7985
9	0,4021	3440,85	0,8269
10	0,3787	3378,57	0,8388

Se seleccionó $k = 3$. Esta decisión se fundamenta en que:

- El Silhouette Score alcanza su máximo (0,5013).
- El índice Davies-Bouldin alcanza su mínimo (0,6852).
- Aunque el índice Calinski-Harabasz es ligeramente superior con $k = 4$, las otras dos métricas favorecen claramente $k = 3$.

3.4.1 Métricas del modelo final

Métrica	Valor
Silhouette Score	0,5013
Calinski-Harabasz	3520,6561
Davies-Bouldin	0,6852

El Silhouette Score de 0,5013 indica una separación moderada y razonable entre los grupos. Los datos presentan una estructura de agrupación identificable, aunque no se trata de clusters perfectamente separados.

3.5 Caracterización de los clusters

Los promedios de las variables originales por cluster son:

Cluster	Close promedio	Volume promedio	Observaciones
0	23 695,51	12 782 580	1 597
1	45 833,34	11 649 830	622
2	15 043,03	28 905 580	631

Interpretación de los grupos:

- **Cluster 0:** Es el grupo más numeroso (1 597 observaciones). Presenta un precio de cierre intermedio y un volumen relativamente bajo. Representa el comportamiento más frecuente del conjunto de datos.
- **Cluster 1:** Compuesto por 622 observaciones. Presenta el precio de cierre promedio más elevado (aproximadamente 45 833) y el volumen promedio más bajo. Corresponde a periodos caracterizados por precios altos y menor actividad de negociación.
- **Cluster 2:** Compuesto por 631 observaciones. Presenta el precio de cierre promedio más bajo (aproximadamente 15 043) y el volumen promedio más elevado (aproximadamente 28,91 millones). Representa periodos de precios bajos acompañados de una actividad de negociación considerablemente mayor.

Los centroides en la escala estandarizada confirman esta separación:

- Cluster 0: Close $\approx -0,244$, Volume $\approx -0,383$
- Cluster 1: Close $\approx 1,603$, Volume $\approx -0,514$
- Cluster 2: Close $\approx -0,965$, Volume $\approx 1,480$

3.6 Gráficas e interpretación

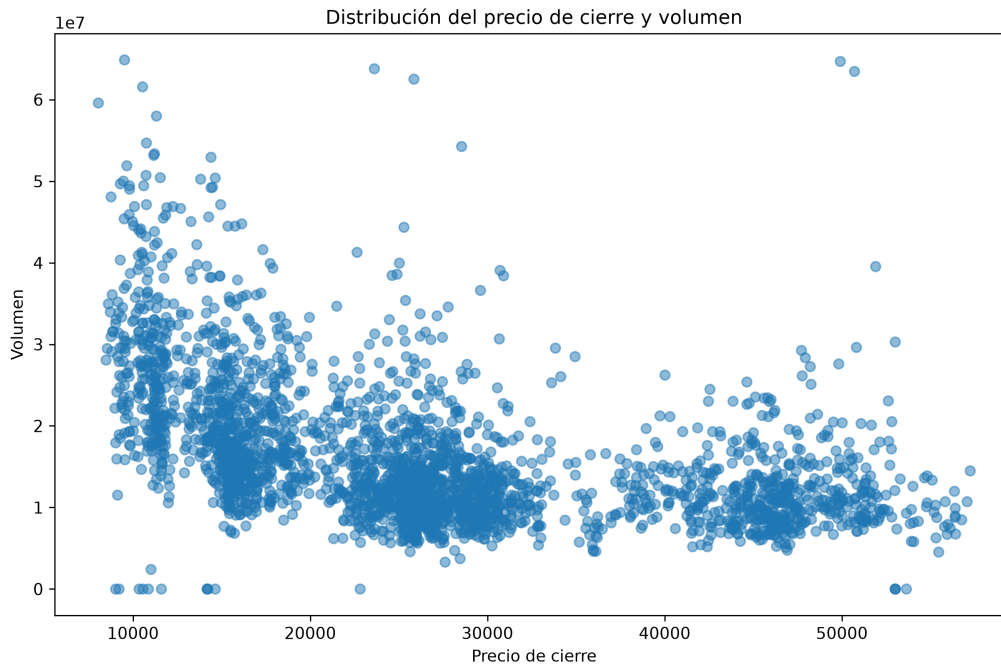


Figura 6: Distribución inicial de las observaciones según precio de cierre y volumen.

La gráfica muestra la distribución de los datos antes de aplicar el algoritmo de agrupación. Se observa una mayor concentración de observaciones en rangos intermedios de precio y volumen, junto con una cola de puntos con volúmenes elevados.

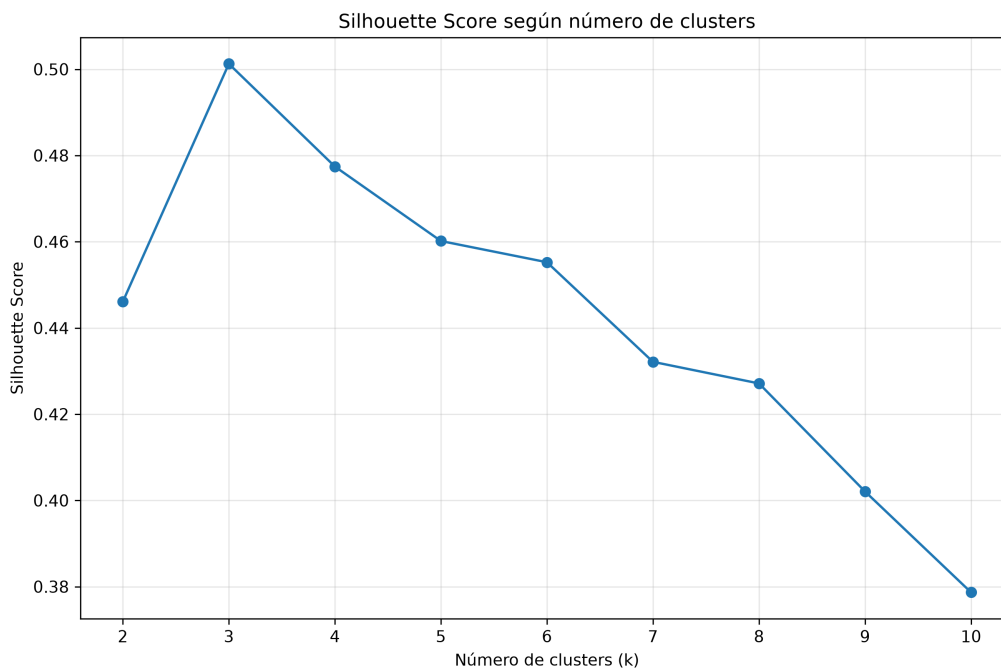


Figura 7: Evolución del Silhouette Score para diferentes valores de k .

El Silhouette Score alcanza su valor máximo en $k = 3$, lo que respalda la selección realizada.

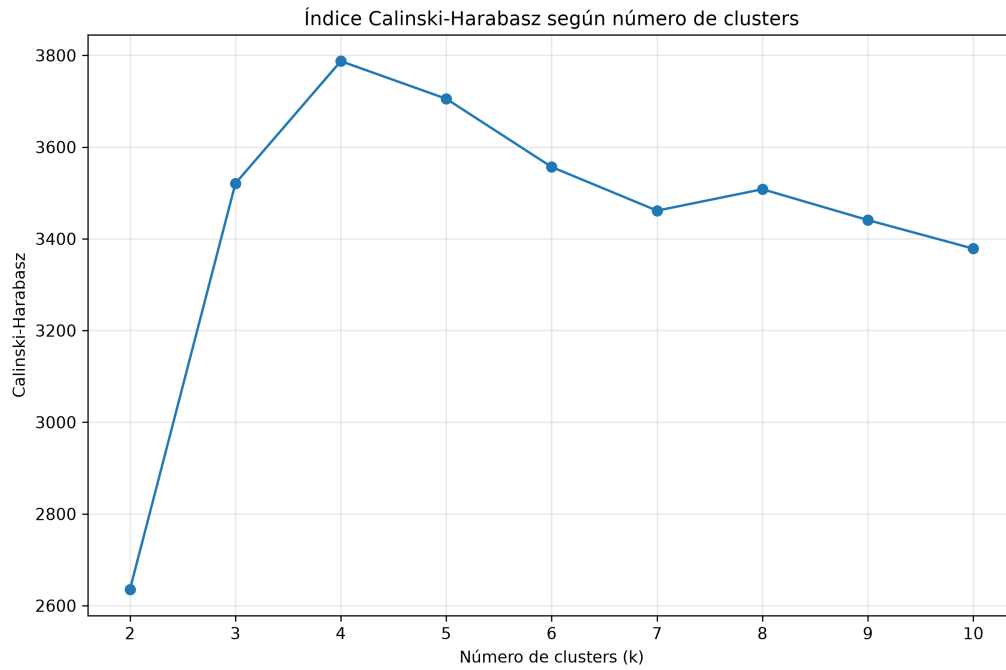


Figura 8: Evolución del índice Calinski-Harabasz para diferentes valores de k .

El índice Calinski-Harabasz presenta su máximo en $k = 4$, aunque el valor obtenido con $k = 3$ también es elevado.

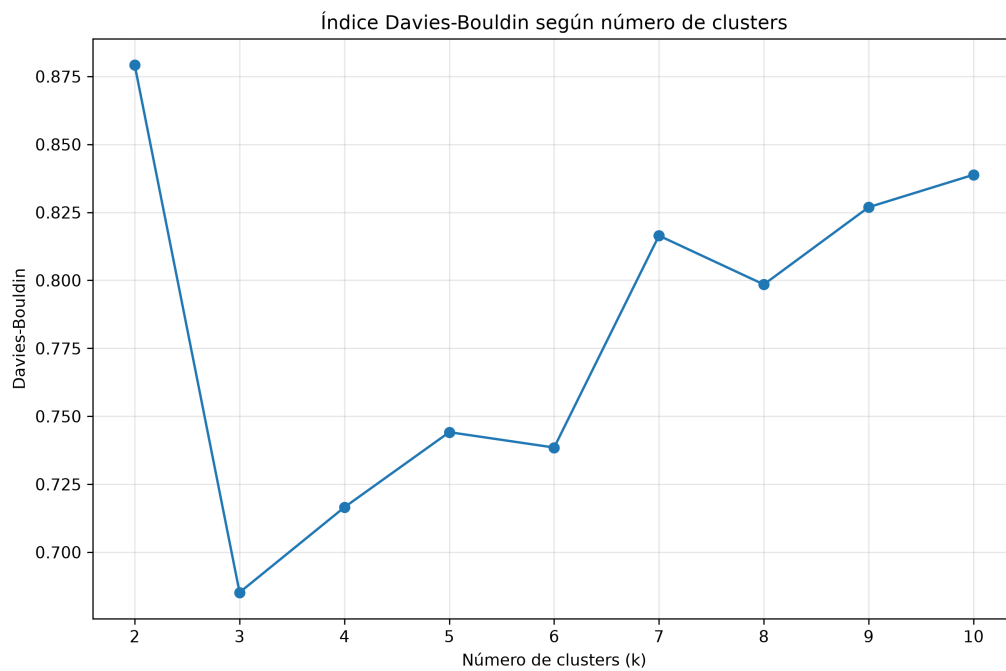


Figura 9: Evolución del índice Davies-Bouldin para diferentes valores de k .

El índice Davies-Bouldin alcanza su valor mínimo en $k = 3$, lo que indica una combi-

nación favorable de cohesión interna y separación entre grupos.

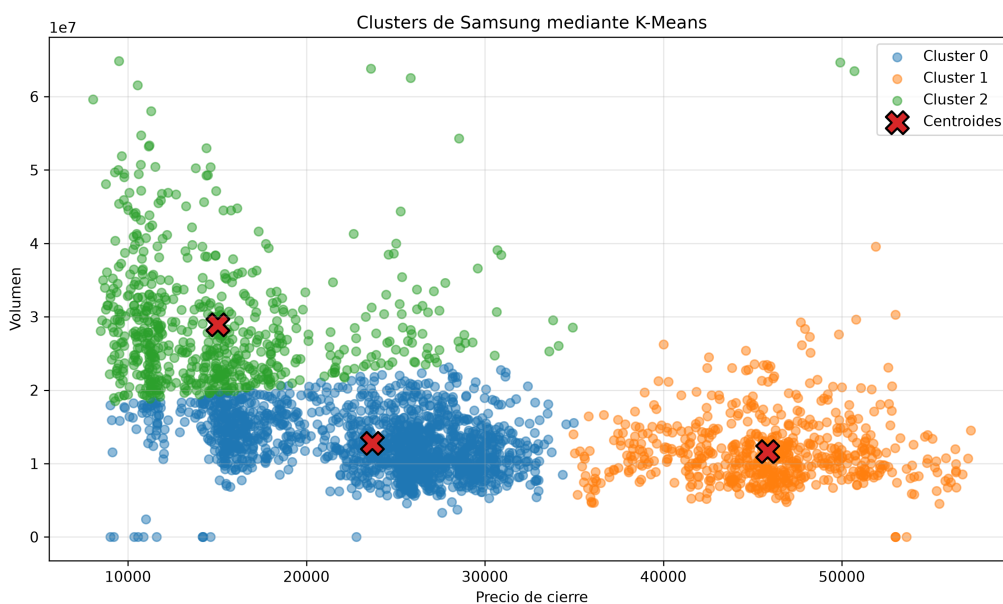


Figura 10: Distribución de las observaciones agrupadas mediante K-Means ($k = 3$) junto con sus centroides.

La gráfica de clusters permite observar tres grupos diferenciados:

- El Cluster 1 se asocia con valores de **Close** considerablemente mayores y niveles relativamente bajos de **Volume**.
- El Cluster 2 se caracteriza por valores de **Close** menores y niveles de **Volume** claramente superiores.
- El Cluster 0 ocupa una posición intermedia y concentra la mayor cantidad de observaciones.

Estas diferencias son consistentes con los promedios calculados para cada grupo y muestran que K-Means identificó patrones diferenciados en el comportamiento conjunto del precio de cierre y el volumen.

3.7 Repositorio del modelo

El código completo del notebook y el modelo final se encuentran disponibles en:

<https://github.com/CarlitrossMM/analisis-supervisado-no-supervisado>

El modelo de agrupación se almacena en la ruta:

```
models/clustering/samsung_kmeans.joblib
```

El objeto guardado corresponde a un Pipeline que integra `StandardScaler` y `KMeans` con $n_clusters = 3$, $random_state = 42$ y $n_init = 10$.

4. Análisis No Supervisado: Reducción de Dimensionalidad con `comprar_alquilar.csv`

4.1 Descripción del problema

El conjunto de datos `comprar_alquilar.csv` contiene 202 registros y 10 variables relacionadas con características económicas y personales. Las variables disponibles son:

Variable	Tipo	Descripción
<code>ingresos</code>	Numérica	Nivel de ingresos
<code>gastos_comunes</code>	Numérica	Gastos comunes
<code>pago_coche</code>	Numérica	Pago relacionado con automóvil
<code>gastos_otros</code>	Numérica	Otros gastos
<code>ahorros</code>	Numérica	Ahorros acumulados
<code>vivienda</code>	Numérica	Valor asociado a la vivienda
<code>estado_civil</code>	Numérica	Estado civil
<code>hijos</code>	Numérica	Número de hijos
<code>trabajo</code>	Numérica	Situación laboral
<code>comprar</code>	Binaria	Variable relacionada con la decisión de compra

La variable `comprar` se identificó como variable objetivo del dataset, pero **no se utilizó en el análisis de componentes principales**. El ejercicio se centró exclusivamente en reducir la dimensionalidad de las nueve variables restantes, conservando la mayor cantidad posible de varianza.

El dataset no presentó valores nulos ni registros duplicados.

4.2 Justificación del algoritmo

Se seleccionó **PCA (Análisis de Componentes Principales)** porque permite transformar un conjunto de variables posiblemente correlacionadas en un nuevo conjunto de componentes ortogonales ordenados según la cantidad de varianza que explican. PCA es especialmente útil cuando se desea reducir la dimensionalidad sin disponer de una variable objetivo para guiar el proceso.

Dado que las variables presentan escalas muy diferentes (por ejemplo, `vivienda` se mide en cientos de miles mientras que `hijos` o `estado_civil` toman valores pequeños), se aplicó `StandardScaler` antes de PCA. De esta forma se evita que las variables con mayor magnitud numérica dominen el cálculo de los componentes.

La optimización del modelo se centró en determinar cuántos componentes conservar, utilizando como criterio la varianza explicada acumulada.

4.3 Diseño del modelo: paso a paso del notebook

A continuación se describen las etapas principales desarrolladas en el notebook.

1. Carga y exploración de los datos.

```
1 df = pd.read_csv("../data/comprar_alquilar.csv")
2
3 print("Dimensiones del dataset:", df.shape)
4 print(df.isnull().sum())
5 print(df.duplicated().sum())
6 print(df.describe())
```

Se confirmaron 202 registros, 10 columnas, ausencia de valores nulos y ausencia de duplicados.

2. Separación de características. Se excluyó la variable `comprar`.

```
1 X = df.drop(columns=["comprar"])
2 print("Forma de X:", X.shape)
3 print(X.columns.tolist())
```

La matriz de características quedó conformada por 202 observaciones y 9 variables.

3. Estandarización.

```
1 scaler = StandardScaler()
2 X_scaled = scaler.fit_transform(X)
```

4. PCA completo y análisis de varianza.

```
1 pca_full = PCA()
2 X_pca_full = pca_full.fit_transform(X_scaled)
3
4 print("Varianza explicada por componente:")
```

```
5 for i, variance in enumerate(  
6     pca_full.explained_variance_ratio_, start=1  
7 ):  
8     print(f"PC{i}: {variance:.4f}")  
9  
10 variance_cumulative = np.cumsum(  
11     pca_full.explained_variance_ratio_  
12 )
```

5. **Selección del número de componentes.** Se evaluó la cantidad de componentes necesaria para alcanzar distintos umbrales de varianza acumulada.

```
1 for threshold in [0.90, 0.95, 0.99]:  
2     n_componentes = np.argmax(  
3         variance_cumulative >= threshold  
4     ) + 1  
5     print(  
6         f"Componentes necesarios para {threshold*100:.0f}% "  
7         f"de varianza: {n_componentes}"  
8     )
```

6. **PCA final con 7 componentes.**

```
1 pca_final = PCA(n_components=7)  
2 X_pca = pca_final.fit_transform(X_scaled)  
3  
4 print("Dimensiones originales:", X.shape)  
5 print("Dimensiones despues de PCA:", X_pca.shape)  
6 print(  
7     "Varianza explicada:",  
8     pca_final.explained_variance_ratio_.sum()  
9 )
```

7. **Análisis de cargas e interpretación de componentes.**

```
1 loadings = pd.DataFrame(  
2     pca_final.components_.T,  
3     index=X.columns,  
4     columns=[f"PC{i}" for i in range(1, 8)]  
5 )
```

8. **Construcción del Pipeline y persistencia.**

```

1 pca_pipeline = Pipeline([
2     ("scaler", StandardScaler()),
3     ("pca", PCA(n_components=7))
4 ])
5
6 pca_pipeline.fit(X)
7
8 joblib.dump(
9     pca_pipeline,
10    "../models/dimensionality_reduction/comprar_alquilar_pca.joblib"
11 )

```

4.4 Selección del número de componentes

La varianza explicada por cada componente y la varianza acumulada fueron las siguientes:

Componente	Varianza individual	Varianza acumulada
PC1	0,2991 (29,91 %)	0,2991 (29,91 %)
PC2	0,2329 (23,29 %)	0,5321 (53,21 %)
PC3	0,1171 (11,71 %)	0,6492 (64,92 %)
PC4	0,1069 (10,69 %)	0,7561 (75,61 %)
PC5	0,0964 (9,64 %)	0,8524 (85,24 %)
PC6	0,0563 (5,63 %)	0,9087 (90,87 %)
PC7	0,0415 (4,15 %)	0,9502 (95,02 %)
PC8	0,0302 (3,02 %)	0,9803 (98,03 %)
PC9	0,0197 (1,97 %)	1,0000 (100 %)

A partir de estos resultados se determinó:

- 6 componentes conservan al menos el 90 % de la varianza.
- 7 componentes conservan al menos el 95 % de la varianza.
- 9 componentes serían necesarios para alcanzar el 99 %.

Se seleccionaron **7 componentes principales**, lo que permite reducir la dimensionalidad de 9 a 7 variables conservando el 95,02 % de la varianza original. Aunque la reducción no es extremadamente grande, elimina dos dimensiones manteniendo prácticamente toda la información estadística relevante.

4.5 Interpretación de los componentes

Las cargas de las variables en los siete componentes principales permiten interpretar el significado de cada dirección:

- **PC1 — Perfil económico:** Las mayores cargas corresponden a `ingresos` (0,4937), `vivienda` (0,4626) y `ahorros` (0,4447). Este componente está principalmente asociado con las variables económicas de mayor peso del hogar.
- **PC2 — Características familiares y laborales:** Destacan `trabajo` (0,5272), `hijos` (0,5219) y `estado_civil` (0,4690). Representa una dimensión relacionada con la situación familiar y laboral.
- **PC3 — Composición de gastos:** Las cargas principales son `gastos_otros` (0,7020), `pago_coche` (−0,5161) y `gastos_comunes` (−0,3635). Refleja un contraste entre gastos adicionales y pagos del automóvil o gastos comunes.
- **PC4 — Gastos de automóvil y otros gastos:** Dominado por `pago_coche` (0,7803) y `gastos_otros` (0,5907).
- **PC5 — Gastos comunes:** La variable dominante es `gastos_comunes` (0,7630), acompañada de una carga negativa de `vivienda` (−0,3656).
- **PC6 — Estado civil:** Determinado principalmente por `estado_civil` (0,7671), con una relación negativa con `hijos` (−0,3880).
- **PC7 — Vivienda, ahorros e hijos:** Combina `vivienda` (0,6410), `ahorros` (−0,5246) e `hijos` (0,4601).

Los signos de las cargas pueden invertirse sin alterar el significado matemático del componente; lo relevante es la magnitud y la relación entre variables.

4.6 Gráficas e interpretación

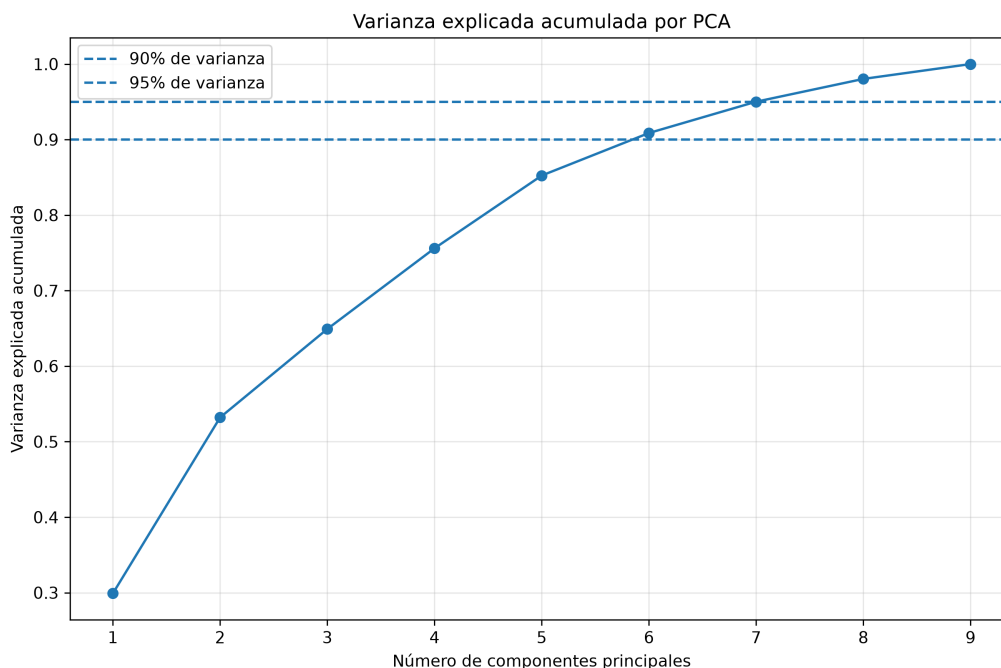


Figura 11: Varianza explicada acumulada por los componentes principales.

La gráfica muestra cómo se acumula la varianza al incorporar componentes sucesivos. Se observa que con 6 componentes se supera el 90 % y con 7 componentes se alcanza el 95 %.

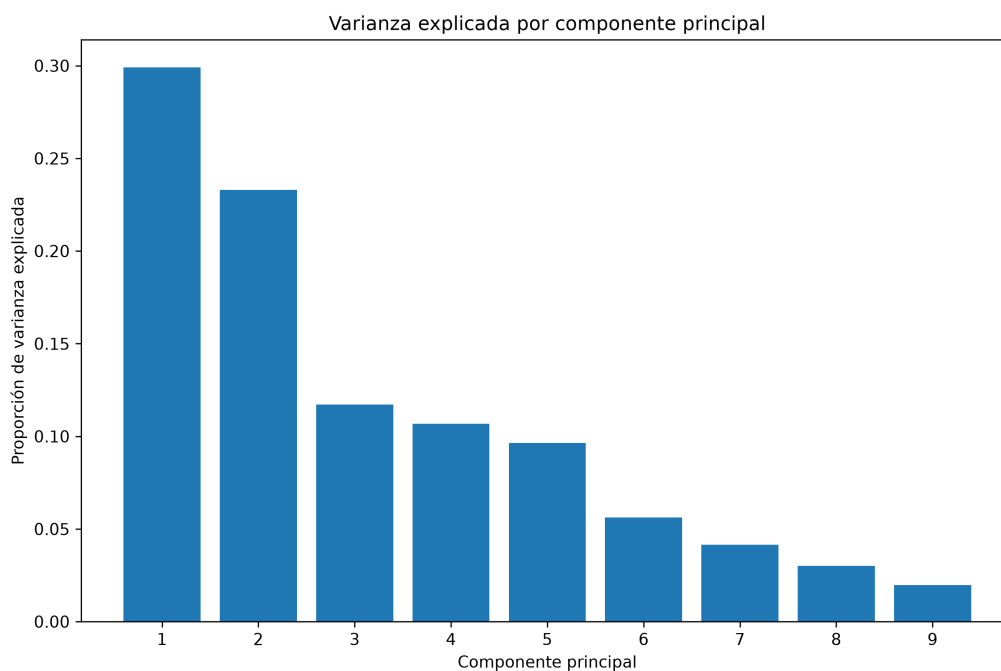


Figura 12: Proporción de varianza explicada individualmente por cada componente.

Los dos primeros componentes concentran más de la mitad de la varianza total (53,21 %), mientras que los componentes posteriores aportan contribuciones decrecientes.

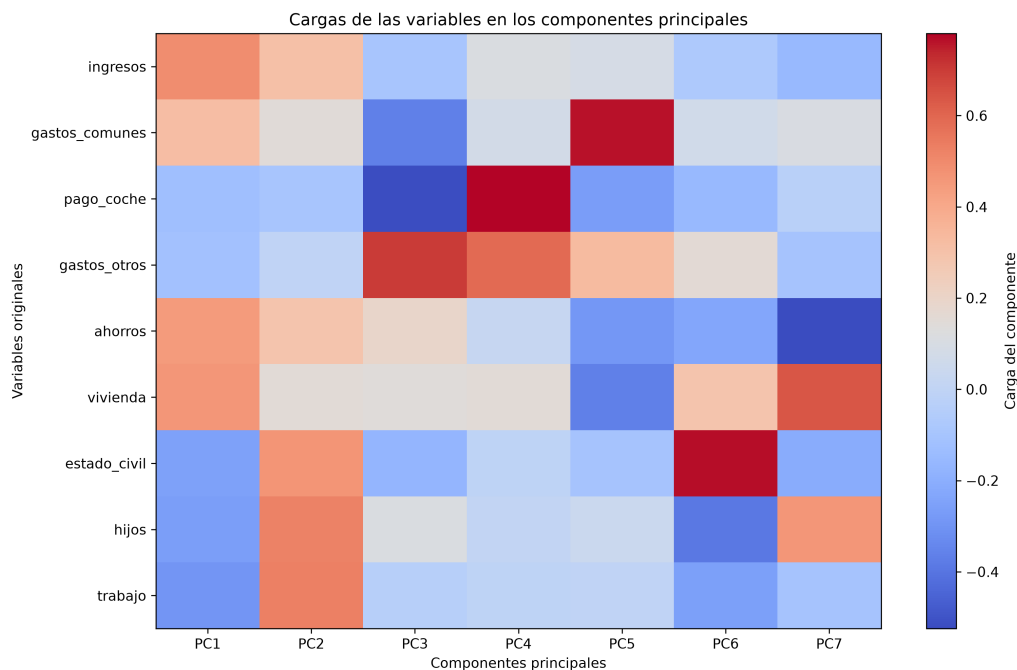


Figura 13: Cargas de las variables originales en los siete componentes principales.

La matriz de cargas facilita la identificación de las variables que más influyen en cada componente y respalda las interpretaciones presentadas anteriormente.

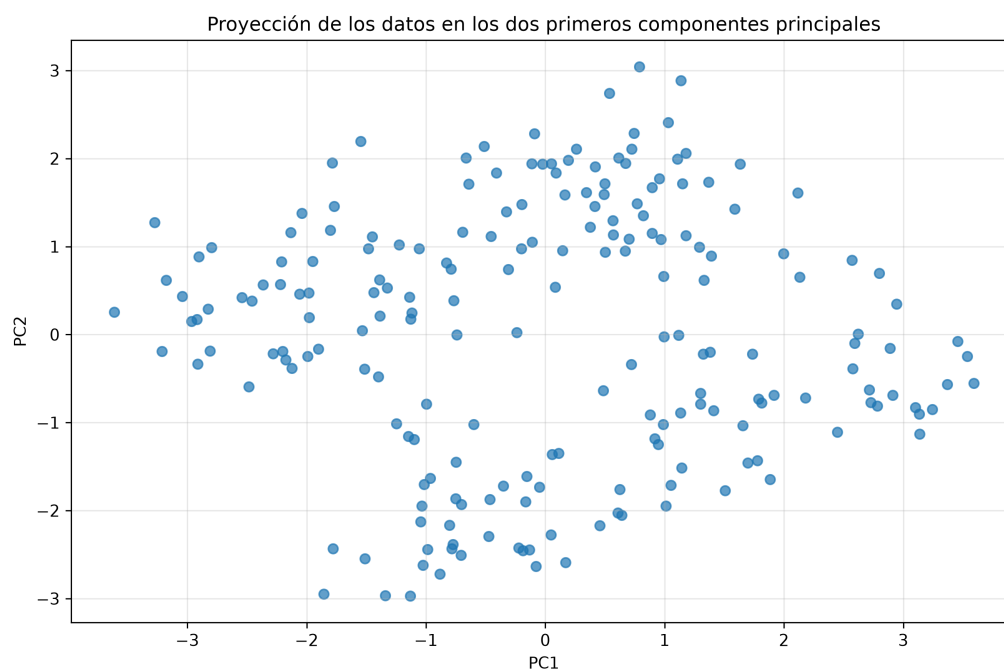


Figura 14: Proyección de las observaciones sobre los dos primeros componentes principales.

La proyección en el plano formado por PC1 y PC2 permite visualizar la distribución de los datos en las dos direcciones de mayor varianza, sin utilizar la variable **comprar** para colorear o condicionar la representación.

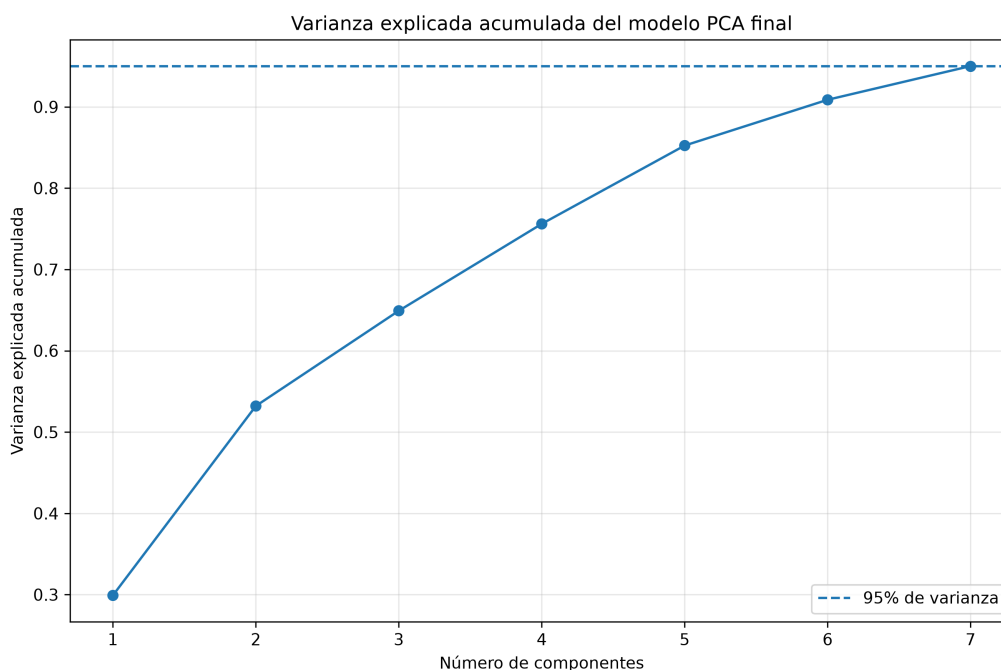


Figura 15: Varianza explicada acumulada del modelo PCA final con 7 componentes.

La gráfica confirma que los siete componentes seleccionados conservan el 95,02 % de la varianza original.

4.7 Repositorio del modelo

El código completo del notebook y el modelo final se encuentran disponibles en:

<https://github.com/CarlitrossMM/analisis-supervisado-no-supervisado>

El modelo de reducción de dimensionalidad se almacena en la ruta:

```
models/dimensionality_reduction/comprar_alquilar_pca.joblib
```

El objeto guardado corresponde a un Pipeline que integra `StandardScaler` y `PCA` con `n_components = 7`. Adicionalmente se almacenaron las cargas de los componentes en:

```
models/dimensionality_reduction/pca_loadings.csv
```

5. Conclusiones generales

A partir del desarrollo de los cuatro ejercicios fue posible aplicar diferentes técnicas de aprendizaje automático de acuerdo con la naturaleza de cada problema.

En el caso de la regresión, la **Regresión Lineal** constituye un modelo inicial adecuado para estudiar la relación entre los bateos y las carreras, mientras que Ridge y Lasso permiten evaluar alternativas regularizadas durante la optimización.

Para clasificación, la **Regresión Logística** proporciona una solución interpretable para distinguir las categorías del conjunto de cáncer de mama. El escalamiento de variables y la optimización del parámetro de regularización permiten mejorar la consistencia del modelo.

En el análisis no supervisado, **K-Means** permite identificar grupos de comportamiento similar en las variables numéricas del conjunto Samsung. La selección de k debe estar respaldada por métricas como la silueta y por la observación de la estructura de los grupos.

Finalmente, **PCA** permite reducir la dimensionalidad del conjunto de compra/alquiler y facilita la visualización de la información en un espacio de menor dimensión. La comparación con y sin escalamiento demuestra que la preparación de variables puede modificar de forma importante los componentes obtenidos.

En conjunto, los experimentos muestran que la elección del algoritmo no debe hacerse únicamente por popularidad, sino considerando el tipo de problema, las variables disponibles, la preparación de los datos, las métricas de evaluación y la posibilidad de interpretar y optimizar los resultados.

Repositorio general

Todo el código fuente, notebooks, modelos y archivos necesarios para reproducir el trabajo se encuentran en:

[https://github.com/CarlitrossMM/
analisis-supervisado-no-supervisado](https://github.com/CarlitrossMM/analisis-supervisado-no-supervisado)

La estructura propuesta del repositorio es:

```
[style=pythonstyle]
analisis-supervisado-no-supervisado/
```

```
|
+-- datasets/
|   +-- beisbol.csv
|   +-- breast_cancer.csv
|   +-- samsung.csv
|   +-- comprar_alquilar.csv
|
+-- notebooks/
|   +-- 01_regresion_béisbol.ipynb
|   +-- 02_clasificacion_breast_cancer.ipynb
|   +-- 03_agrupacion_samsung.ipynb
|   +-- 04_reduccion_dimensionalidad_comprar_alquilar.ipynb
|
+-- models/
|   +-- regression/
|   +-- classification/
|   +-- clustering/
|   +-- dimensionality_reduction/
|
+-- figuras/
|
+-- README.md
+-- requirements.txt
```